Project Notes: Development of Modern Statistical Techniques for LLMs and Prediction

Kevin Wang

April 22, 2026

Contents

1 Introduction **2**

2 AI History and the Development of Predictive Models **2**

2.1 A broad pattern . . . 2
2.2 From symbolic order to learned structure . . . 3
2.3 From randomness to separability: the emergence of neural networks . . . 3
2.4 Neural networks . . . 3
2.5 First-generation neural networks: the perceptron . . . 4
2.6 Second-generation neural networks: early multi-layer perceptrons . . . 5
2.7 Why this stage matters . . . 7
2.8 The continued development of neural networks . . . 7
2.9 LSTM and GRU: improving long-range dependence . . . 8
2.10 CNN . . . 8
2.11 Structural change in neural networks: from recurrence to attention . . . 9
2.12 Transformer: a new paradigm of fully parallel sequence modeling . . . 10
2.13 Early language models before the full LLM era . . . 12
2.14 Toward the pretraining paradigm . . . 13
2.15 BERT as the first decisive breakthrough in the new paradigm . . . 14
2.16 OpenAI and the GPT series . . . 14
2.16.1 GPT-1 . . . 14
2.16.2 GPT-2 . . . 15
2.16.3 The organizational shift: from nonprofit research to capped-profit structure . 16
2.16.4 GPT-3 and ChatGPT . . . 16
2.16.5 GPT-4 . . . 17
2.16.6 A second restructuring and the long-run tension . . . 17
2.17 The layered evolution of OpenAI's business model . . . 17
2.18 Flagship products and the coupling of technology with use cases . . . 18
2.19 The self-reinforcing logic of the ecosystem . . . 18
2.20 Other large-scale models and the open-source push . . . 19
2.21 Core trends: compute, parameters, data, and beyond . . . 19

1 Introduction

Making predictions and drawing inference are among the most important application areas of statistics. Modern machine learning, and especially large language models, can be understood as a continuation of this broader statistical project: given increasingly large and unstructured data, how can we construct models that recover structure, generalize to unseen inputs, and produce useful predictions.

The goal of this project is to understand how the complex predictive models we see today came to take their current form. In particular, I focus on large language models and related architectures for prediction. Rather than treating current models as isolated engineering artifacts, I want to place them in a longer historical and conceptual development. By tracing how earlier statistical and machine learning techniques evolved, one can better understand why modern architectures look the way they do, what problems each stage was trying to solve, and what kinds of progress became possible once particular modeling or computational bottlenecks were broken.

This project is written for an audience with some mathematical foundation. Accordingly, the emphasis is not merely on chronology, but on the internal logic of the development itself: how the field moved from explicit logical systems to learned representation, from linear separation to nonlinear approximation, and eventually toward large-scale architectures for prediction over high-dimensional structured data.

This set of notes are compiled with the help of AI as an overview structure of the project, as a result of the preliminary research and reading. The final form will be a website that contains a timeline, key historical events in the development of AI and prediction algorithms, important mathematical proofs, as well as interactive examples showcasing how the most important models work.

2 AI History and the Development of Predictive Models

2.1 A broad pattern

The history of artificial intelligence may be read as a sequence of attempts to answer one common question: how can disordered, high-dimensional, and unstructured data be turned into a form from which meaningful prediction becomes mathematically tractable.

Over time, the dominant answers changed. Early symbolic systems attempted to impose order through explicit logic and hand-coded rules. Neural networks shifted the emphasis from manually specified reasoning to data-driven representation learning. Later architectures further modified how information was stored, transmitted, and extracted, allowing models to handle more complex forms of dependence and larger scales of data.

From this perspective, the development of AI is not just a list of models, but a gradual change in the way prediction itself is approached. Each new paradigm emerged because the previous one could no longer adequately balance three things: expressive power, trainability, and scalability.

2.2 From symbolic order to learned structure

The early development of AI was driven by symbolic systems and expert systems such as MYCIN and DENDRAL. The core idea was to use a fixed logical framework to perform reasoning, and to represent knowledge through explicit rules and manually constructed knowledge bases. Under this view, intelligence was treated primarily as rule-based symbolic manipulation.

This approach had a certain internal appeal. If reasoning can be reduced to formal logic, then one may hope to build intelligent systems by writing down the relevant rules and letting the system deduce conclusions from them. In practice, however, symbolic systems encountered a fundamental difficulty. The real world contains a large amount of tacit knowledge, contextual dependence, and implicit structure. Human beings are able to ignore irrelevant variables and focus on what matters through intuition and background understanding, but symbolic systems require all relevant conditions to be explicitly encoded. As a result, the rule set expands rapidly, computation becomes intractable, and the system loses flexibility and real-time usefulness.

This is closely related to the classical frame problem. Symbolic systems attempted to solve intelligence through logical axiomatization, but in realistic environments knowledge is fragmented, context-dependent, and difficult to fully formalize. The more complex the environment, the harder it becomes to maintain such a system. As a result, symbolic AI remained narrow in scope, difficult to scale, and limited in audience and application.

From a broader modeling perspective, symbolic AI put too much of the burden on prior explicit specification. It sought to impose order from above. The next paradigm instead attempted to learn structure from data directly.

2.3 From randomness to separability: the emergence of neural networks

As manually constructing intelligence through rules reached a dead end, an alternative approach gradually took the lead. Instead of prescribing intelligence through logical rules, one could attempt to simulate how learning itself works. This was the central intuition behind connectionism. Inspired by the structure of biological neural systems, connectionist models abandoned explicit rule enumeration and took data as the starting point, constructing networks that learn internal parameters from examples.

This shift marks one of the major conceptual transitions in the history of predictive modeling. The main problem was no longer how to write all relevant rules by hand, but rather how to build a flexible system capable of extracting useful structure from data. In modern terms, this is the turn from rule-based reasoning to learned function approximation.

2.4 Neural networks

A neural network is a computational model inspired by biological nervous systems. It mimics, at a very abstract level, the way neurons process information. The model consists of interconnected artificial neurons arranged in layers: an input layer that receives raw signals, one or more hidden layers that transform and abstract features, and an output layer that produces predictions. Each neuron computes a weighted sum of its inputs and then applies a nonlinear activation function, such as ReLU, thereby mimicking the threshold-like response behavior of biological neurons.

In current practice, the training process adjusts these weights automatically through backpropagation. One may think of this, loosely, as analogous to synaptic adaptation in the brain: the model modifies its internal connections in response to observed data so as to better fit the predictive task. Although neural networks borrow the hierarchical and adaptive flavor of biological systems, they remain fundamentally different from the human brain in feedback efficiency, energy use, and interpretability. Still, over time, their architecture has become increasingly powerful.

Early neural network structures were quite primitive. They began with simple computational layers and later developed into models with multiple hidden layers. With this development, the paradigm of AI changed. The field moved away from attempting to construct all rules directly and toward approximating the learning process itself.

At this point, the central problem of data-driven machine learning became the following: given chaotic, heterogeneous data, how can one transform it into a form that is separable enough for classification or prediction to become possible. In this sense, neural networks are not merely computational tools. They are mechanisms for reshaping data into more mathematically manageable representations.

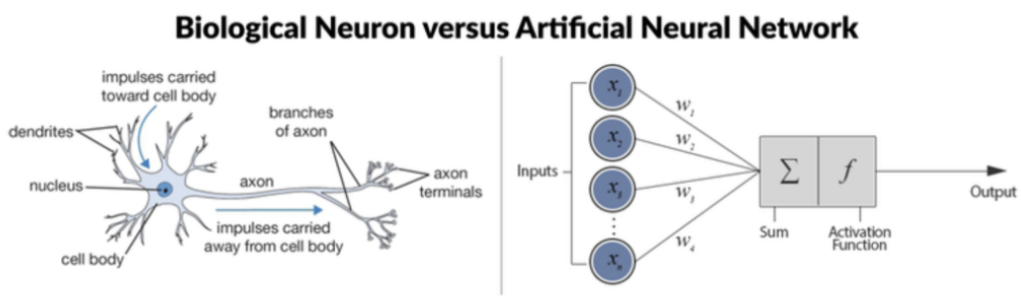

Figure 1: Neural Networks

2.5 First-generation neural networks: the perceptron

The earliest major step in this direction was the perceptron, first proposed by Rosenblatt in 1957. The perceptron is a single-layer linear classifier that can handle simple binary linearly separable problems, such as deciding between yes and no, or classifying an observation into class A or class B. It was an early attempt to simulate a basic biological neuron: receiving input, forming a weighted sum, and activating if the sum exceeds a threshold.

Its basic mathematical form is

$$f(x) = \operatorname{sgn}(w^\top x + b)$$

The perceptron was historically important because it established an early model of automatic learning. Instead of relying on manually coded rules, the model learns internal parameters from labeled data. In this sense, it represents a decisive turn from symbolic AI toward connectionism.

At the time, the perceptron generated enormous enthusiasm. It seemed to suggest that learning machines could indeed be built. But its success came with a severe limitation. Because it relies on a linear decision function, it can only solve linearly separable problems. It fails on fundamentally nonlinear tasks, the classic example being XOR. More generally, low-dimensional linear separation

cannot represent complex class structure. A single line cannot separate two concentric rings, and a single hyperplane cannot express more intricate nonlinear boundaries.

Thus, while the perceptron was an important conceptual breakthrough, it remained trapped inside the limitations of linear classification. It could serve as an initial exploration of neural learning, but it could not support the richer forms of intelligence people hoped for.

Note: A dataset $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{-1, 1\}$ is linearly separable if there exist w, b such that

$$y_i(w^\top x_i + b) > 0 \qquad \text{for all } i$$

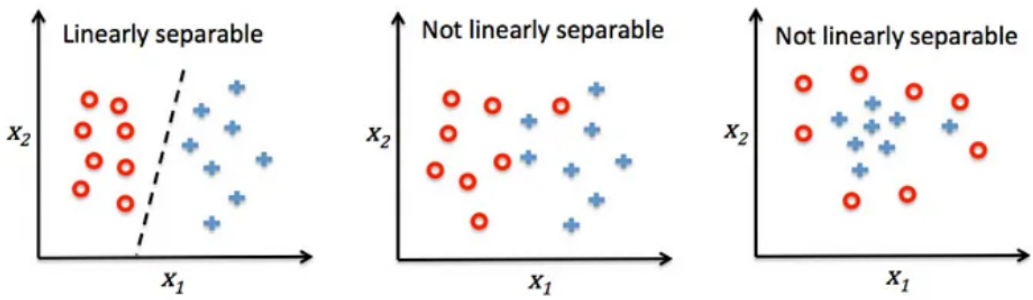

Figure 2: perceptron / linear separability

2.6 Second-generation neural networks: early multi-layer perceptrons

To solve the limitations of linear separability, Ivakhnenko in 1965 proposed the multi-layer perceptron. Its core structure defined the basic paradigm of later neural networks. By introducing hidden layers and nonlinear activation functions, the model can transform the feature space itself. Each hidden layer learns progressively more abstract and complex representations, and through this layered transformation the model can construct nonlinear decision boundaries.

A simplified mathematical description is

$$h^{(1)} = \sigma(W_1 x + b_1), \qquad h^{(2)} = \sigma(W_2 h^{(1)} + b_2), \qquad y = W_3 h^{(2)} + b_3$$

More generally, a depth-L feedforward network may be written as

$$f(x) = W_L \sigma(W_{L-1} \sigma(\cdots \sigma(W_1 x + b_1) \cdots) + b_{L-1}) + b_L$$

The essential idea is that each layer no longer merely classifies the original data. Instead, it produces a new representation of the data, and later layers operate on that transformed representation. In this way, the model uses nonlinear feature mapping to make difficult classification problems more manageable.

The multi-layer perceptron includes an input layer, one or more hidden layers, and an output layer, with adjacent layers fully connected. This architecture became the basic foundation from which later neural architectures developed. Convolutional neural networks, recurrent neural networks, and Transformer-based models all inherit this broader principle of layered learned representation, though they modify the connectivity pattern and computational modules substantially.

Early MLPs, however, remained difficult to train. In their primitive form, training often required regression-style procedures, manual checking of unit importance, removal of redundant units, and dynamic adjustments to the number of hidden layers and neurons based on training error. This demanded large amounts of human labor and could not support large datasets well.

Over the following two decades, machine learning gradually addressed these limitations. The cumbersome nature of early MLP training, together with the difficulty of extracting complex features from large data, became one of the main bottlenecks the field had to solve. Contributions by Amari (1967), Linnainmaa (1970), and later Rumelhart and Hinton (1986) were crucial in the development of backpropagation. This created the algorithmic basis for training increasingly large neural networks and laid important groundwork for the later GPU era of large-scale model expansion.

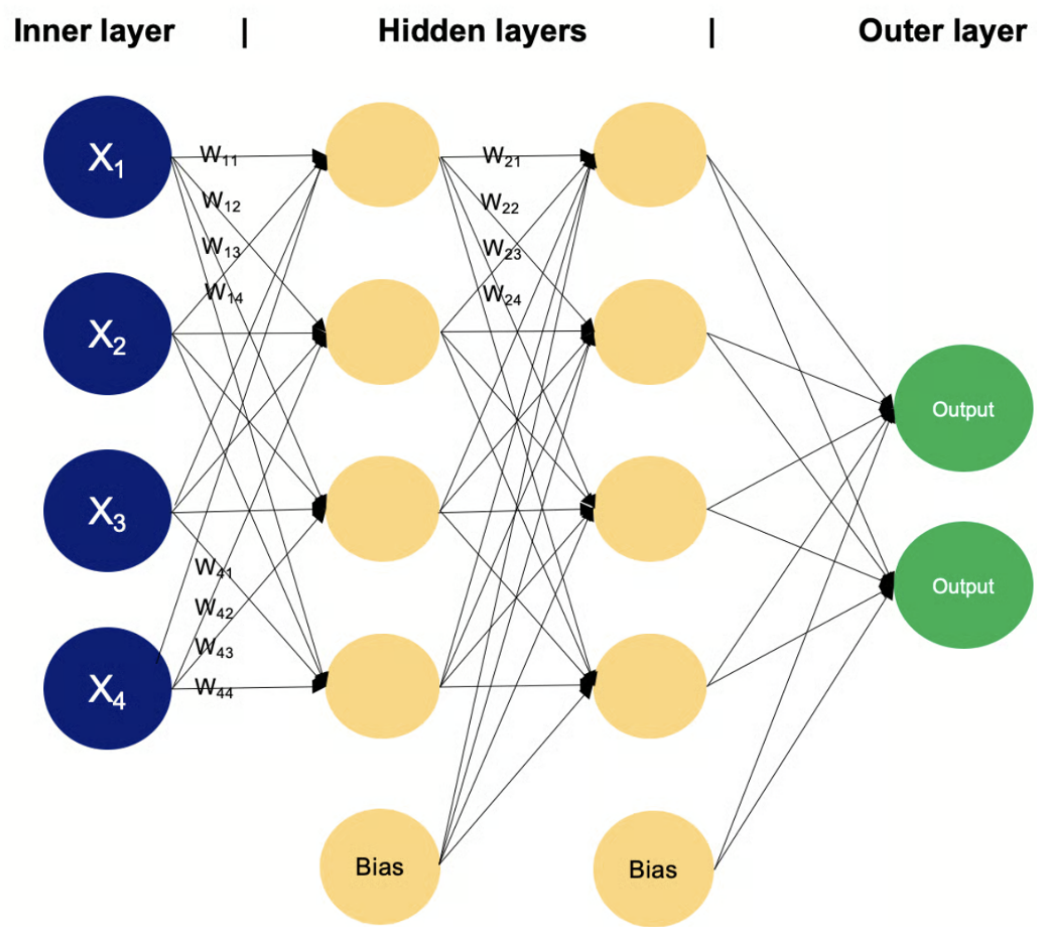

Figure 3: Multi-Layer Perceptron Architechture

A bit more mathematical details on backpropagation: For $z^{(\ell)} = W^{(\ell)}h^{(\ell-1)} + b^{(\ell)}$ and $h^{(\ell)} = \sigma(z^{(\ell)})$,

$$\delta^{(\ell)} = \frac{\partial L}{\partial z^{(\ell)}} = \left((W^{(\ell+1)})^\top \delta^{(\ell+1)}\right) \odot \sigma'(z^{(\ell)})$$

2.7 Why this stage matters

This first stage in the history of modern AI established the central statistical logic that continues into contemporary large models. The field began with attempts to impose explicit order through logic, then moved toward learning structure from data. Within neural networks themselves, it began with linear separation and then progressed toward multilayer nonlinear approximation.

The importance of this transition is not merely historical. It set the terms for everything that followed. Once intelligence was reframed as the problem of learning useful internal representation from data, the path was opened toward deeper architectures, more complex dependence structures, and eventually the large-scale predictive systems that define the current era of machine learning.

2.8 The continued development of neural networks

After the early development of multi-layer perceptrons, neural networks continued to evolve in order to address more structured forms of data. In recent decades, recurrent neural networks and their variants rose rapidly in natural language processing. The core motivation was straightforward: many important data types, such as text, speech, and other sequential observations, cannot be adequately modeled by architectures that treat each input as isolated. What matters is not only the current input itself, but also the order in which information appears and the way earlier information influences later prediction.

The earliest recurrent neural network, often referred to as the vanilla RNN, consists of an input layer, a hidden layer, and an output layer, with recurrent hidden-to-hidden connections transmitting information serially across time. At each time step, the RNN combines the current input with the previous hidden state, applies an activation function, and produces a new hidden state. In this way, it can remember short-term context and process streaming sequential data online.

A simplified recurrence may be written as

$$h_t = \phi(W_x x_t + W_h h_{t-1} + b)$$

The main conceptual contribution of the RNN is that it introduces temporal state into neural modeling. It allows the model to accumulate historical information and use that evolving internal state to make later predictions. In this sense, it breaks the limitation of standard feedforward networks, which assume fixed-dimensional independent inputs.

Historically, the core idea of recurrent modeling had already been laid out between the 1970s and 1980s. In 1972, Amari, building on Ising-style ideas, proposed one of the earliest recurrent structures. In 1982, the Hopfield network showed how recurrent architectures could support stable-state computation. In 1986, the Elman network and Jordan network formed prototypes of the modern trainable RNN. But because of insufficient computational power and the difficulty of optimization, recurrent networks remained relatively limited for a long time. Only around 2013, with the explosion of GPU computation and the practical success of improved recurrent units, did RNNs experience large-scale revival and become widely used from 2013 to 2015.

Models based on RNNs, such as the RNN language model proposed by Mikolov and later sequence-to-sequence models such as Sutskever's machine translation system, used recurrent structure to probabilistically model sequential data. These models significantly improved the ability

to capture context dependence over longer ranges and laid important foundations for machine translation, speech recognition, and event prediction.

At the same time, the classical RNN had major limitations. Because each step depends on the previous hidden state, the model cannot be parallelized across time steps. More importantly, long-distance dependence is hard to learn because gradients either vanish or explode over long sequences. In practical terms, the model often loses useful information after roughly a limited number of steps and struggles to preserve long-run memory.

2.9 LSTM and GRU: improving long-range dependence

To solve the long-distance dependence problem in recurrent models, Hochreiter and Schmidhuber proposed the long short-term memory network, or LSTM. The LSTM extends the standard RNN by adding a more structured memory mechanism. Its key components include

(1) a forget gate, which removes irrelevant historical information

(2) an input gate, which selectively adds new information into memory

(3) an output gate, which controls how much current information is passed forward

(4) a cell state, which allows gradients to travel through a more stable path

The importance of the LSTM is that it fundamentally alleviates the vanishing gradient problem. Instead of forcing all information through a fragile chain of nonlinear transitions, it creates a more persistent memory channel. Beginning around 2013, LSTMs almost dominated sequence modeling, and were widely used in tasks such as machine translation, speech synthesis, and extreme-event prediction.

Later, Cho et al. (2014) proposed the gated recurrent unit, or GRU, as a simpler and more efficient variant. The GRU merges some of the gates in the LSTM, in particular combining the input and forget mechanisms into a single update gate, while also retaining a reset gate to control the interaction between old and new information. As a result, the parameter count is smaller and training is often faster, especially on medium-scale or resource-constrained tasks. In many practical applications, the GRU performed competitively with the LSTM while being simpler to implement and train.

The broader lesson of this stage is that once neural networks were extended to sequential data, the central challenge became not just representation, but also memory and information flow. RNNs introduced the idea of temporal dependence, while LSTMs and GRUs refined the way relevant information is preserved and irrelevant information is discarded.

2.10 CNN

Convolutional neural networks developed along a different direction and achieved major breakthroughs in computer vision. Their main advantage comes from local receptive fields and parameter sharing, which allow them to extract spatial features efficiently. In later years, improvements such as ReLU, Dropout, and GPU-based parallel training made CNNs highly effective and dominant in many visual tasks. Although CNNs are not the main focus here, they are important as part of the

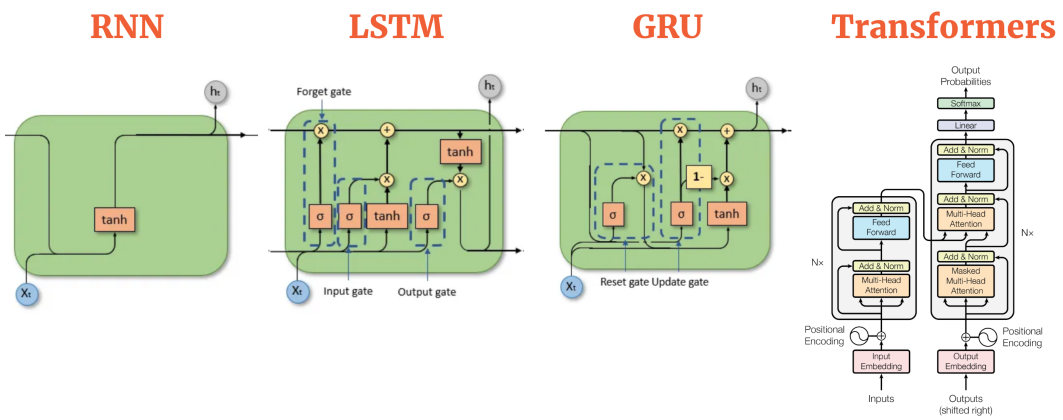

Figure 4: RNN, LSTM, and GRU comparison

broader development of neural network structure: different architectures emerged because different data types demanded different inductive biases and computational tradeoffs.

2.11 Structural change in neural networks: from recurrence to attention

Although RNNs and LSTMs introduced memory into sequence modeling, they still suffered from a major bottleneck. In long text or long sequences, the model often had to compress the relevant information into a fixed-dimensional hidden representation. This compression easily caused loss of important information, especially when the input was long and information-dense.

Against this background, Bahdanau et al. (2014) proposed the attention mechanism. The intuition is very natural: a model should not treat all input information equally at all moments. Instead, like a human reader, it should dynamically focus on the parts of the input most relevant to the current task. In other words, rather than forcing all information into one bottleneck vector, the model is allowed to revisit different positions of the input and weight them differently depending on what it is currently generating.

Technically, the attention mechanism computes a relevance score for each position in the input sequence. These scores are based on a similarity function between the decoder state, which reflects what the model is currently trying to produce, and the encoder states, which reflect the content of the input sequence. A SoftMax then turns these scores into a weight distribution. The final context vector is a weighted average of the encoder hidden states.

Compared to earlier recurrent architectures, this mechanism has several major advantages:

(1) **Long-range access**: information at any input position can be accessed directly without passing through many recurrent steps

(2) **Dynamic computation**: each output step can generate its own attention weights and focus on different parts of the input

(3) **Interpretability**: the weights can be visualized, giving a partial explanation of what the model is attending to

(4) **Generality**: attention is modular and can be added to RNNs, CNNs, and other architectures

In this sense, attention changed the core statistical logic of sequence modeling. The problem was no longer purely one of transmitting information through time. It became a problem of dynamically learning relevance across positions in a structured input.

A useful comparison is the following:

Dimension	Traditional RNN / Seq2Seq	Attention mechanism	Main breakthrough
Information storage	Fixed-length vector, which compresses and distorts information	Sequence of hidden states, with one state for each input position	Breaks the dimensional bottleneck
Long-range access	Gradient decay, effective range limited	Direct weighted access across positions	Alleviates long-distance information loss
Interpretability	Largely black-box	Attention weights can be visualized	Makes decisions more transparent
Hardware efficiency	Serial computation, low GPU utilization	Matrix-based parallel computation	Greatly improves computational efficiency

Still, early attention mechanisms did not completely eliminate recurrence. They continued to rely on RNNs to generate encoder states, and they also depended on previous decoder outputs. So while attention broke the fixed-vector bottleneck, it did not yet fully remove the temporal chain in the computation. It was, however, the decisive transitional step.

2.12 Transformer: a new paradigm of fully parallel sequence modeling

Building on the attention mechanism, Vaswani et al. introduced the Transformer architecture in 2017 in *Attention Is All You Need*. This architecture solved three core limitations of earlier models.

First, it replaced the time-step dependent recurrence of RNNs with stacked self-attention and feedforward layers. The encoder-decoder computation could now be carried out through matrix operations, rather than by serially stepping through time.

Second, it addressed the information overload problem of a single attention path by introducing multi-head attention, allowing the model to focus on different kinds of structure simultaneously.

Third, it fully reconstructed sequence computation into a global matrix-based form, thereby releasing the parallel power of modern hardware.

The Transformer consists of an encoder and a decoder, each built from repeated stacked sublayers. Each sublayer contains the following main modules:

(1) **Multi-head self-attention**: instead of one single attention pattern, the model uses multiple attention heads operating in different subspaces. One head may focus more on syntactic

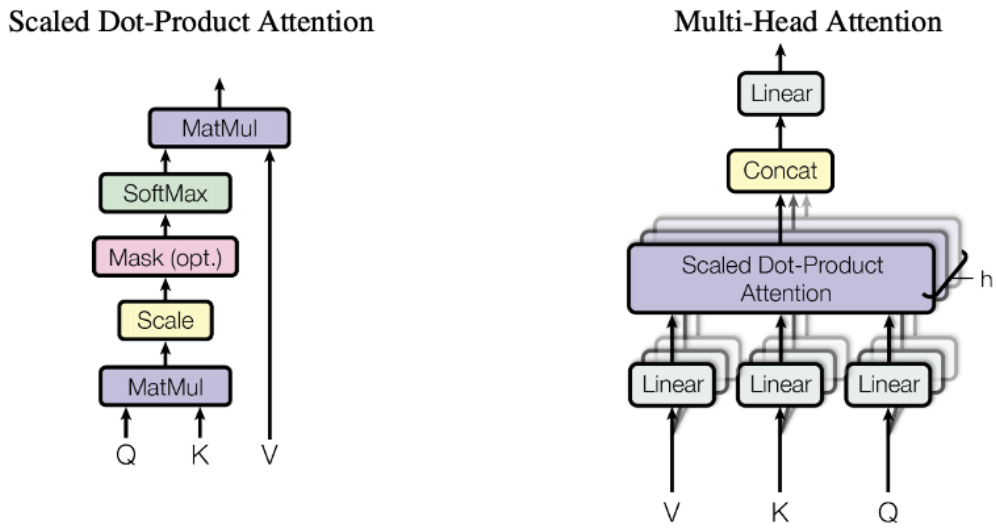

Figure 5: attention mechanism, linking to the multi-head attention mechanism of transformer

alignment, another on semantic connection, and another on positional or dependency structure.

(2) **Feed-forward network**: each position is further transformed by the same fully connected network, increasing local representational power.

(3) **Positional encoding**: because the Transformer has no recurrence built in, order information must be injected explicitly. One standard form is

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

with a corresponding cosine term for odd dimensions.

(4) **Residual connection and layer normalization**: these improve optimization stability and help prevent gradient problems in deeper architectures.

The most disruptive innovation of the Transformer is the reconstruction of sequence computation into a parallel paradigm. A traditional RNN must update hidden states step by step, so sequence length directly constrains the computational pathway. By contrast, the Transformer treats the sequence as a globally connected structure and computes relationships among positions through parallel matrix multiplication.

This changed the field in two ways. Technically, it removed the serial bottleneck. Statistically, it made it possible to learn richer contextual relations over long sequences at much larger scale. Once training could be parallelized efficiently across modern hardware, much deeper and larger models became feasible.

In practical terms, this hardware alignment mattered enormously. Query, key, and value transformations in multi-head attention can be distributed across large numbers of GPU threads. This

made the Transformer dramatically faster than recurrent architectures for large-scale training, and it quickly became the indispensable foundation for the era of large pre-trained models.

One may summarize the difference simply:

- RNNs move information forward through time
- attention directly connects relevant positions
- Transformers make this process fully parallel and scalable

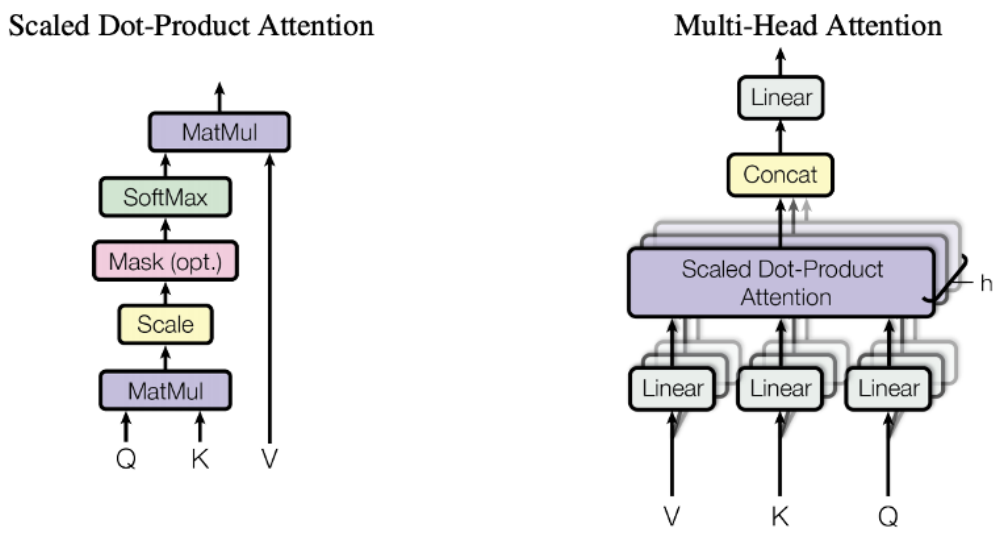

Figure 6: Transformer and Attention Mechanism

The Transformer also proved highly extensible. It became the backbone of later pre-trained models such as BERT and the GPT series, and eventually expanded beyond language into multimodal systems such as DALL-E and architectures in vision such as ViT. In this sense, Transformer did not simply replace RNNs inside NLP. It became a general computational language for modern AI.

Attention and Transformer should not be viewed as isolated inventions. They are best understood as systematic responses to long-standing problems in recurrent modeling. Attention weakened the compression bottleneck. Transformer completed the move by removing recurrence itself. Each step made information flow more flexible, computation more efficient, and scaling more practical. This is why deep learning was finally able to enter a genuinely large-scale and general-purpose stage.

2.13 Early language models before the full LLM era

Before the emergence of modern large language models, the main attempts at language modeling were much more limited in scale and structure. One of the earliest dominant approaches was the n-gram model. Based on a Markov assumption, an n-gram language model predicts the next word using the frequencies of short preceding word fragments.

These models were widely used in early machine translation and speech recognition because they were simple to implement, transparent in interpretation, and fast in both training and inference. But they ran into severe sparsity problems. In a large vocabulary space, most higher-order combinations of words appear rarely or not at all. Directly estimating probabilities from counts therefore becomes unreliable. The combinatorial explosion of possible contexts outpaces the observed sample size.

Researchers also attempted to build neural language models earlier using multi-layer perceptrons. In such models, one-hot vectors over vocabularies of size ten thousand to one hundred thousand were mapped into hidden layers and then used for prediction. But because the vocabulary space was so large and the possible combinations so numerous, these early approaches were computationally difficult and did not scale well.

A major turning point came around 2003, when larger unlabeled text corpora, GPU acceleration, and distributed training frameworks gradually matured. Bengio et al. proposed one of the first neural network language models. The key idea was to place an embedding layer at the front of the model, mapping high-dimensional one-hot vectors into dense lower-dimensional representations. This replaced extremely sparse count-based tables with continuous vector spaces containing only tens or hundreds of dimensions. As a result, the model generalized better and handled local context much more effectively.

Later, Word2Vec in 2013 further improved efficiency. Rather than attempting full probabilistic language modeling in the earlier NNLM sense, Word2Vec focused on learning static word embeddings through a shallow network and efficient training objectives such as negative sampling or hierarchical SoftMax. Instead of optimizing a full probability distribution over the entire vocabulary at every step, it trained on word-context pairs. This made learning much faster while still producing strong initial vector representations. These embeddings later became important building blocks for more sophisticated language models.

2.14 Toward the pretraining paradigm

Early unsupervised pretraining ideas had already been explored by Hinton and others around 2006. At that time, the goal was to train restricted Boltzmann machines or autoencoders layer by layer, thereby giving deep networks a better initialization and alleviating problems such as local optima and vanishing gradients. But this form of bottom-up pretraining was cumbersome. It required each module to be trained separately and then stacked, which imposed substantial cost.

As activation functions such as ReLU, initialization schemes such as Xavier and He initialization, and optimizers such as Adam improved, and as larger labeled datasets became available, this earlier style of pretraining gradually became less central. In NLP in particular, RNN-based language model pretraining remained constrained by sequential computation and therefore could not scale efficiently to very large parameter counts. In addition, the objective of early autoencoder-style pretraining often did not align closely with downstream tasks such as translation or reading comprehension, which limited the value of fine-tuning.

The Transformer changed this. Because self-attention allows the model to compute over all positions in parallel, pretraining on massive corpora became much more practical. Entire sentences or even larger text spans could be processed at once, fully using the parallel power of GPUs and TPUs. More importantly, the predictive objectives used in Transformer-based pretraining aligned

much more naturally with downstream language tasks. This created the real opening for the modern pretraining era.

2.15 BERT as the first decisive breakthrough in the new paradigm

A major milestone in this transition was BERT in 2018. BERT used a bidirectional Transformer encoder and was pre-trained with objectives such as masked language modeling and next sentence prediction. The model could then be fine-tuned on downstream NLP tasks and quickly surpassed prior state-of-the-art results across a wide range of benchmarks.

The importance of BERT lies not just in its empirical performance, but in what it represented conceptually. It showed that large-scale pretraining on general language objectives could produce highly transferable representations. Rather than building separate architectures from scratch for each task, one could first learn a broad language model and then adapt it.

This marks the endpoint of the stage covered here. By this point, the field had already moved through several major transformations: from symbolic order to learned structure, from feedforward networks to recurrent memory, from recurrence to attention, and from task-specific modeling toward large-scale pretraining. The next stage is the rise of GPT and the full large language model era.

2.16 OpenAI and the GPT series

If BERT represented the decisive breakthrough of the encoder-style pretraining paradigm, then OpenAI and the GPT series pushed the field toward a different and ultimately even more commercially explosive direction. The core shift was from bidirectional representation learning toward autoregressive generation. Rather than masking parts of the input and reconstructing them, GPT models predict the next token from left to right. This design gives up full bidirectional conditioning, but it aligns much more naturally with generation itself. In that sense, GPT is not merely another large model. It reflects a different view of what a language model should do.

OpenAI itself initially began as a technology-driven and explicitly non-commercial organization. In 2015, it was founded by Sam Altman, Elon Musk, Peter Thiel, and others, with a core team that included Ilya Sutskever and researchers from Google Brain. The organization was structured as a nonprofit research lab, with a charter centered on the safe development of artificial general intelligence. At that stage, the emphasis was on open research, technical progress, and safety, rather than on productization or monetization. Funding relied primarily on donations, and the early work focused on foundational capability rather than immediate commercial application.

2.16.1 GPT-1

GPT-1, released in 2018, was the first major validation of this path. It used a Transformer decoder architecture with unidirectional autoregressive pretraining and had roughly 117 million parameters. The model was trained on a large corpus of books and demonstrated that a single generative pretraining objective, followed by relatively light task-specific fine-tuning, could transfer well across many language tasks.

The technical importance of GPT-1 lies in the fact that it established the "generative pretraining + fine-tuning" paradigm. By using masked self-attention, the model generates text from left to right, so each token prediction depends only on the left context. This sacrifices some bidirectional

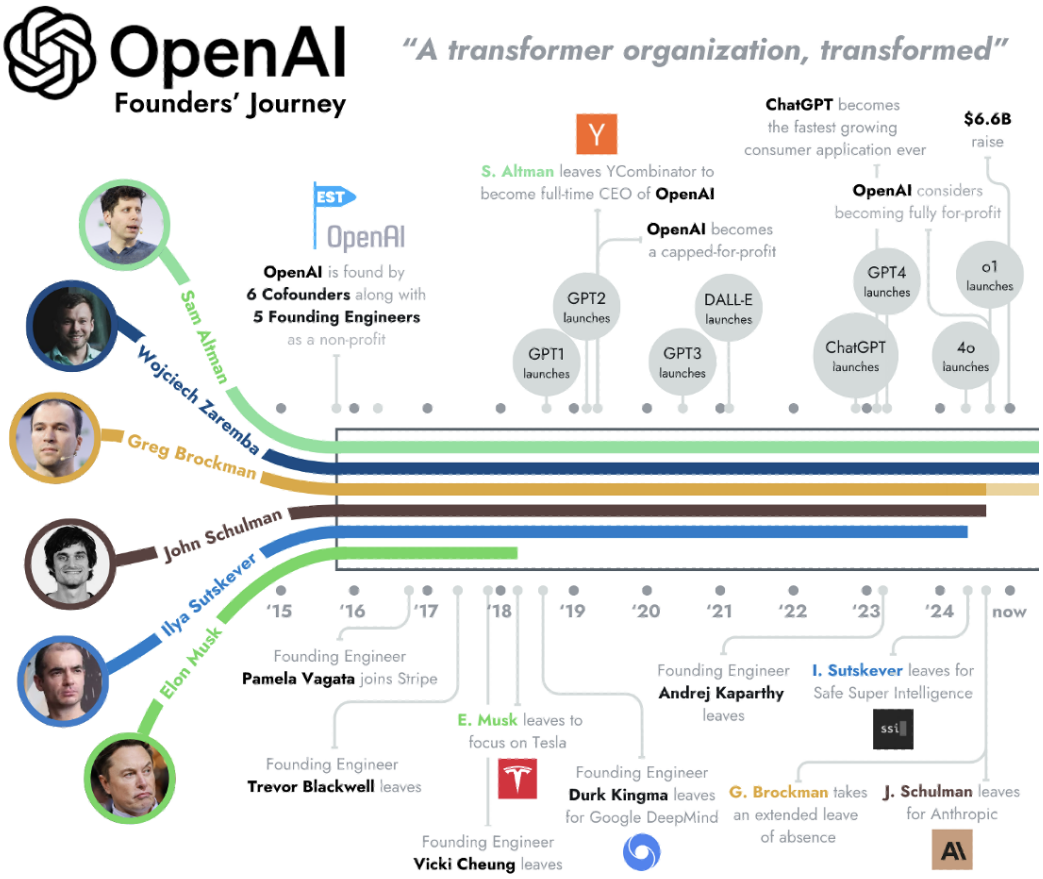

Figure 7: Company, products, and the founders

semantic access, but it is highly consistent with the structure of text generation itself. GPT-1 showed that a sufficiently large autoregressive model could learn broadly useful representations without needing a bespoke architecture for every task.

2.16.2 GPT-2

GPT-2 in 2019 marked a much larger jump. The parameter count increased to 1.5 billion, and the training corpus expanded dramatically to large-scale web text. More importantly, GPT-2 displayed striking zero-shot and few-shot abilities. Instead of requiring explicit fine-tuning for every task, the model could often perform new tasks simply from prompts and examples placed in the context window.

This was a major conceptual moment. It suggested that once model scale crossed a certain threshold, a new kind of generalization began to emerge. The model no longer merely fit a narrow predictive distribution. It started to internalize more abstract task patterns embedded in large internet text corpora. For example, by giving the model a short translation pattern such as "English to French: hello → bonjour," it could continue the pattern without any task-specific retraining.

Technically, this suggested that the combination of Transformer architecture, large-scale pre-training, and scale in both data and parameters was beginning to unlock qualitatively new behav-

ior. GPT-2 brought language generation and language understanding into a much more public and practical form.

2.16.3 The organizational shift: from nonprofit research to capped-profit structure

By 2018–2019, the training cost of large models such as GPT-2 had already begun to rise sharply. Training frontier models now required not just good ideas, but enormous computational infrastructure and capital. This created tension for OpenAI's original nonprofit structure. In 2019, OpenAI therefore restructured and created OpenAI LP, adopting a capped-profit model.

This structure was unusual. It allowed outside investment and employee equity incentives, but imposed a cap on returns, while keeping formal control in the hands of the nonprofit parent. In principle, this was meant to preserve the mission of benefiting humanity while still attracting the capital required for large-scale AI development. The structure quickly enabled Microsoft's billion-dollar investment and created the now familiar cycle of capital injection, cloud infrastructure support, and technical output. In practical terms, this was a recognition that frontier AI had already become inseparable from large-scale compute economics.

2.16.4 GPT-3 and ChatGPT

GPT-3 in 2020 pushed the autoregressive scaling strategy much further, reaching 175 billion parameters. The most important result was not merely the increase in raw performance, but the strengthening of in-context learning. With only a few examples in the prompt, GPT-3 could adapt to a wide variety of tasks, including writing, translation, summarization, and coding-related tasks.

This suggested that the prompt itself could function as a lightweight task-conditioning mechanism. Instead of updating parameters, one could steer the model through context. The model seemed to infer task structure from examples and instructions placed in the input. This made the system far more flexible and significantly broadened its possible uses.

Still, GPT-3 also had obvious weaknesses. Its generations could be fluent but factually incorrect, biased, or misaligned with user intent. Raw generative power alone did not solve the problem of useful interaction.

ChatGPT in 2022 addressed this by introducing a different technical and product logic. Based on the GPT-3.5 family, it incorporated reinforcement learning from human feedback, or RLHF, to improve instruction-following, safety, and dialogue usefulness. The broad procedure included three stages:

(1) supervised fine-tuning on human-written instruction-response pairs

(2) preference modeling, in which a reward model learns human ranking preferences over outputs

(3) reinforcement learning, in which the model is optimized to produce responses with higher learned preference scores

The importance of RLHF is that it moved the model away from being merely a broad next-token predictor and toward being a usable assistant. In other words, the objective changed from general language continuation to interaction aligned with human intent and preference. This was the step that made large models dramatically more compelling to the mass public. ChatGPT

reached one hundred million users within an exceptionally short time and transformed AI from a largely technical subject into a mainstream product phenomenon.

2.16.5 GPT-4

GPT-4, released in 2023, represented another major increase in capability, although exact parameter counts were not publicly disclosed. Its importance was not only in stronger reasoning and broader robustness, but also in multimodal ability, especially the integration of text and image input. More importantly, GPT-4 came bundled with a broader product vision: plugins, tool use, external function calling, and increasingly flexible interaction with outside systems.

This matters because it marks another conceptual transition. The model is no longer just a text generator or even a conversational interface. It begins to act more like a general-purpose reasoning and orchestration layer, connected to tools, databases, and broader software systems.

2.16.6 A second restructuring and the long-run tension

As OpenAI's commercial footprint expanded, the earlier capped-profit arrangement came under growing strain. Once ChatGPT scaled globally and valuations rose sharply, the tension between capital requirements, commercial growth, and governance became harder to contain. The well-known leadership turmoil in late 2023 exposed this tension clearly.

The deeper issue is structural. Building frontier models increasingly requires massive capital expenditures, especially for compute. At the same time, these organizations often retain a public-facing mission centered on safe AGI and broad benefit. The governance problem is therefore not incidental. It is built into the economics of the technology itself.

2.17 The layered evolution of OpenAI's business model

OpenAI's business model can be understood as a staged monetization of model capability. As model quality improved, the company progressively translated this capability into multiple layers of revenue.

The first major commercial engine was the API business. After GPT-1 and especially GPT-3 had validated the technical promise of large generative models, OpenAI began exposing model access through paid cloud APIs. Under this structure, the model becomes a standardized service that developers can call on demand. This was important because it turned general model capability into something modular and commercially distributable. Rather than only building one end-user product, OpenAI could become infrastructure for many downstream products.

A second layer came from consumer subscription. ChatGPT's popularity made it possible to extend from infrastructure to direct consumer monetization through products such as ChatGPT Plus. This changed the business from a purely developer-facing model into a hybrid one that served both end users and enterprise demand.

A third layer came from enterprise products. ChatGPT Enterprise focused on larger organizations that required stronger privacy, higher throughput, dedicated support, and integration into existing workflows. This was especially important in industries such as finance, healthcare, legal services, and software, where the willingness to pay is much more closely tied to workflow productivity, compliance, and error tolerance.

A fourth layer came from deeper ecosystem integration, especially through Microsoft's platform. OpenAI's relationship with Microsoft was not merely a financing arrangement. It created an entire channel through which the models could be embedded into widely used enterprise software such as Office and GitHub. This dramatically expanded distribution and made the models part of existing work environments rather than standalone curiosities.

From this perspective, OpenAI's commercial model is not based on one single revenue stream. It is layered:

(1) model API services

(2) subscription products

(3) enterprise solutions

(4) licensing, partnerships, and ecosystem integration

The significance of this layered structure is that it allows model improvements to be monetized at multiple levels simultaneously. Better capability improves consumer subscription value, enterprise willingness to pay, API demand, and platform partnership value all at once.

2.18 Flagship products and the coupling of technology with use cases

OpenAI's products follow a fairly clear internal logic: base model capability first, then interface, then scenario deepening.

At the base is the model family itself. GPT-4 and related systems provide the core general capability. GPT-3.5, for example, served for a period as an accessible entry point for broad user adoption, while more advanced models anchored premium features and higher-value applications.

On top of the models sits the interface layer, with ChatGPT as the most important example. ChatGPT drastically lowered the barrier to AI use. It translated a technically complex model into an interface almost anyone could use through plain language. This mattered enormously for adoption. A strong model without an accessible interaction form remains limited in reach. The chatbot format turned model capability into a mass-use product.

Beyond this came deeper product specialization. Codex and its descendants extended the model into software development. DALL-E extended it into image generation. Whisper pushed into speech recognition and voice-related tasks. Over time, these products became less like isolated tools and more like a connected toolchain. One product could feed into another. For instance, a user might draft text, generate images, write code, and process audio all within a connected ecosystem.

This is commercially important because it creates demand nesting. A single user need can trigger multiple model-based services. Once the ecosystem becomes sufficiently integrated, the company is no longer simply selling a model. It is operating a broader AI environment.

2.19 The self-reinforcing logic of the ecosystem

The broader commercial ecosystem around OpenAI operates like a flywheel. Free or low-cost products attract users and developers, increasing familiarity and experimentation. Paid subscriptions and enterprise products then monetize the higher-value segments. The resulting revenue supports

even more expensive model training and infrastructure expansion, which then improves the products further.

This loop is strengthened by product complementarity. A user who adopts one model-driven workflow is more likely to adopt adjacent ones if they live in the same environment. The same underlying model improvements can propagate across text, code, image, and speech applications. As a result, the competitive barrier is no longer just raw model quality. It becomes the combination of model quality, product integration, distribution, and ecosystem lock-in.

This is why the commercialization of modern LLMs cannot be understood purely as a technical story. It is a joint story about model architecture, data, compute, product form, and distribution.

2.20 Other large-scale models and the open-source push

Although OpenAI played the most visible role in popularizing large language models, it is not the only major actor. Other frontier-scale models, including Google's PaLM, DeepMind's Gopher, Megatron-Turing NLG, and BLOOM, showed that very large-scale pretraining was not unique to one organization. At the same time, open-source models such as Meta's LLaMA, Mistral, Falcon, and StableLM pushed rapid iteration outside the fully closed ecosystem.

This is important because the field has increasingly split between open and closed strategies. Closed models often maintain an advantage in scale, integration, and product polish, while open models accelerate experimentation, customization, and diffusion into narrower domains. In practice, this means that the future landscape is unlikely to be monopolized by one single model family. Instead, it will likely involve coexistence between a small number of dominant general platforms and a broader layer of specialized or open systems.

2.21 Core trends: compute, parameters, data, and beyond

At the most general level, the history traced here points to several persistent trends.

First, progress has repeatedly depended on interactions between algorithmic ideas and hardware change. Better architecture alone is rarely sufficient. Major advances have happened when model structure and computational infrastructure align. This was true for backpropagation and GPUs, for CNNs and parallel vision workloads, for Transformers and matrix-based accelerator hardware, and it remains true for today's frontier models.

Second, scale continues to matter. Larger datasets, more parameters, and more compute have repeatedly unlocked qualitatively different behavior. But the relevant question is no longer only brute-force scale. Increasingly, the problem is how to use scale efficiently.

This is why techniques such as sparse attention and mixture-of-experts are important. They try to lower the effective compute cost while preserving or even improving capability. In other words, the field is moving from simple scaling toward more compute-efficient scaling.

Third, model capability increasingly depends on the integration of external knowledge and tools. Pure parametric memory is often insufficient, especially when factual precision and verifiability matter. The combination of retrieval, external databases, and tool use with language models reflects a broader movement toward systems that are not only fluent, but more grounded and operationally useful.

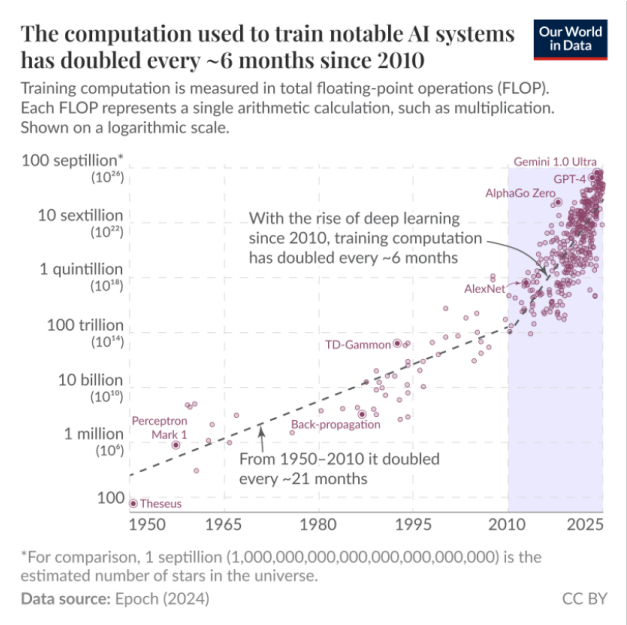

Figure 8: Compute and Development Progress

Fourth, AI is moving beyond pure text. Speech, image, video, and even 3D environments are becoming part of the same general modeling framework. This is not merely an add-on. It suggests that the long-run direction is toward more general interactive systems rather than isolated text predictors.

Some conclusions:

From the earliest symbolic systems to today's large language models, the development of AI can be understood as a sequence of attempts to solve increasingly difficult problems of representation, prediction, and scale.

Symbolic AI tried to impose order explicitly through logic, but proved too brittle and difficult to scale. Neural networks shifted the field toward learning from data, first through linear separation and then through layered nonlinear representation. Recurrent networks introduced temporal dependence and memory, but were constrained by serial computation and fragile long-range learning. Attention relaxed the bottleneck by allowing dynamic relevance across positions, and Transformers completed the shift by making sequence modeling parallel, scalable, and far more compatible with modern hardware. Once this happened, large-scale pretraining became practical, and models such as BERT and GPT revealed that broad language prediction could produce highly transferable internal structure.

The rise of OpenAI and the GPT series then showed that these advances were not only technically significant, but also commercially transformative. The frontier of AI is now shaped not just by statistical modeling in the narrow sense, but by the interaction of model design, compute, data, productization, and ecosystem control.